# The Developers Arena

# Week 7 Task

**Submitted by punit kumar, Batch (10-jan-2026 - 3M)**

# Week 7 – STATISTICAL-ANALYSIS-REPORT – Project Documentation

## Statistical Analysis Report

### Project Overview

This project aims to apply fundamental statistical concepts used in data science to analyze customer-related business data and derive meaningful insights. The dataset consists of 500 customer records containing information such as tenure, monthly charges, total charges, contract type, payment method, senior citizen status, and churn indicator

The key objectives of this project are

- To summanze customer data using descriptive statistics
- To analyze the distribution of business variables
- To identify relationships between customer attributes using correlation analysis
- To evaluate business assumptions using hypothesis testing
- To estimate population parameters using confidence intervals
- To assess predictive relationships using regression analysis

By interpreting the statistical results in a business context, the project demonstrates how data-driven analysis supports strategic decision-making, customer retention planning, and pricing optimization,

## Setup Instructions

The project was developed and executed using Python in Visual Studio Code (VS Code). The following stops were followed to set up the environment and run the analysis:

- Python version 3.x was used as the programming environment
- The dataset file (7.csv) was placed in the same directory as the Python script
- The main analysis script was saved as week7_analysis.py
- Required libraries were installed using pip, including:
  - pandas for data manipulation
  - numpy for numerical computation
  - matplotlib and seaborn for data visualization
  - scipy for statistical testing
  - scikit-learn for regression analysis

● The script was executed through the VS Code integrated terminal using the command: python week7_analysis.py

Successful execution of the script generated numerical outputs and visualizations required for analysis and reporting.

### Code Structure

The code is organized in a logical and sequential manner to ensure clarity, readability, and ease of evaluation. The structure of the script is as follows:

- Data Loading and Inspection
  - Importing required libraries
  - Reading the CSV dataset
  - Displaying data preview and data types

- Descriptive Statistics Section
  - Calculation of mean, standard deviation, minimum, maximum, and quartiles
  - Summary of tenure, monthly charges, and total charges

- Data Distribution Analysis
  - Histogram visualization of monthly charges
  - Shapiro-Wilk test to check normality

- Correlation Analysis
  - Calculation of Pearson correlation coefficients
  - Heatmap visualization for key numerical variables

- Hypothesis Testing
  - Independent t-tests for group comparisons
  - One-way ANOVA for multi-group comparison
  - Interpretation based on p-values

- Confidence Interval Estimation
  - Calculation of 95% confidence interval for monthly charges

- Regression Analysis
  - Linear regression using tenure and monthly charges
  - Evaluation using R-squared value

Each section of the code is clearly labeled using comments, making the analysis easy to follow and modify if required.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.linear_model import LinearRegression

df = pd.read_csv("business_data.csv")
df.columns = df.columns.str.strip()

df.head()
df.info()
```

[8] Python

```
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Date         100 non-null    str
 1   Product      100 non-null    str
 2   Quantity     100 non-null    int64
 3   Price        100 non-null    int64
 4   Customer_ID  100 non-null    str
 5   Region       100 non-null    str
 6   Total_Sales  100 non-null    int64
dtypes: int64(3), str(4)
memory usage: 5.6 KB
```

```python
df[['Quantity', 'Price', 'Total_Sales']].describe()
```

[9] Python

| | Quantity | Price | Total_Sales |
|---|---|---|---|
| count | 100.000000 | 100.000000 | 100.000000 |
| mean | 4.780000 | 25808.510000 | 123650.480000 |
| std | 2.588163 | 13917.630242 | 100161.085275 |
| min | 1.000000 | 1308.000000 | 6540.000000 |
| 25% | 2.750000 | 14965.250000 | 39517.500000 |
| 50% | 5.000000 | 24192.000000 | 97955.500000 |
| 75% | 7.000000 | 38682.250000 | 175792.500000 |
| max | 9.000000 | 49930.000000 | 373932.000000 |

```python
plt.hist(df['Total_Sales'], bins=20)
plt.title("Total Sales Distribution")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")
plt.show()

stats.shapiro(df['Total_Sales'])
```

[10] Python

Generate + Code + Markdown | Run All Clear All Outputs | Outline ··· Python 3.14.2

| | Quantity | Price | Total_Sales |
|---|---|---|---|
| Quantity | 1.000000 | 0.008014 | 0.688107 |
| Price | 0.008014 | 1.000000 | 0.646131 |
| Total_Sales | 0.688107 | 0.646131 | 1.000000 |

[12]

```python
regions = df['Region'].unique()
regions
```

Python

```
<StringArray>
['East', 'North', 'West', 'South']
Length: 4, dtype: str
```

[13]

```python
region_a = df[df['Region'] == regions[0]]['Total_Sales']
region_b = df[df['Region'] == regions[1]]['Total_Sales']

stats.ttest_ind(region_a, region_b)
```

Python

```
TtestResult(statistic=np.float64(-0.35710677600784724), pvalue=np.float64(0.7226811638407538),
```

[14]

```python
groups = [df[df['Region'] == r]['Total_Sales'] for r in regions]
stats.f_oneway(*groups)
```

Python

```
F_onewayResult(statistic=np.float64(2.164363429897842), pvalue=np.float64(0.09723678084979993)
```

```
F_onewayResult(statistic=np.float64(2.164363429897842), pvalue=np.float64(0.09723678084979993)
```

```python
high_qty = df[df['Quantity'] > df['Quantity'].median()]['Total_Sales']
low_qty = df[df['Quantity'] <= df['Quantity'].median()]['Total_Sales']

stats.ttest_ind(high_qty, low_qty)
```

[15] Python

```
TtestResult(statistic=np.float64(7.779538586652192), pvalue=np.float64(7.470163657941178e-12),
```

```python
mean = df['Total_Sales'].mean()
std = df['Total_Sales'].std()
n = len(df)

margin_error = stats.t.ppf(0.975, n-1) * (std / np.sqrt(n))
(mean - margin_error, mean + margin_error)
```

[16] Python

```
(np.float64(103776.34767080679), np.float64(143524.6123291932))
```

Generate + Code + Markdown

```python
X = df[['Quantity', 'Price']]
y = df['Total_Sales']

model = LinearRegression()
model.fit(X, y)

model.score(X, y)
```

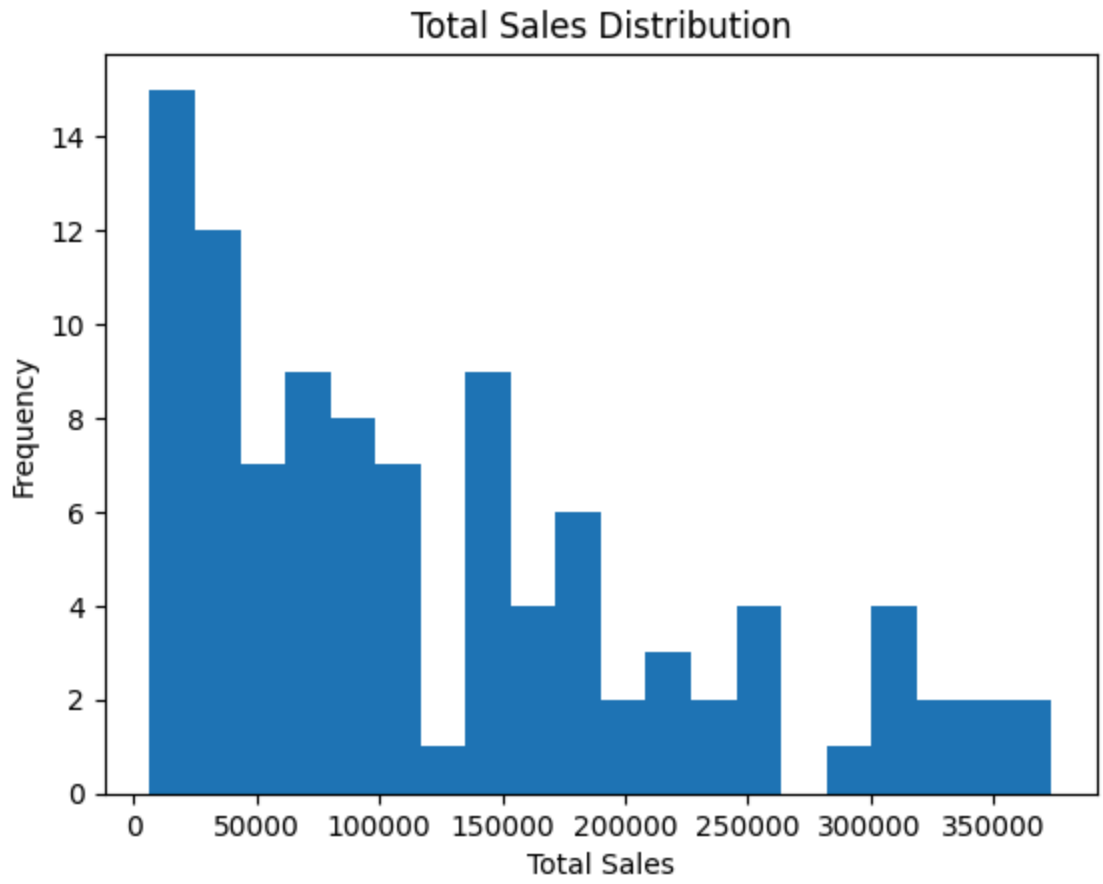
[17] Python

## Visual Documentation

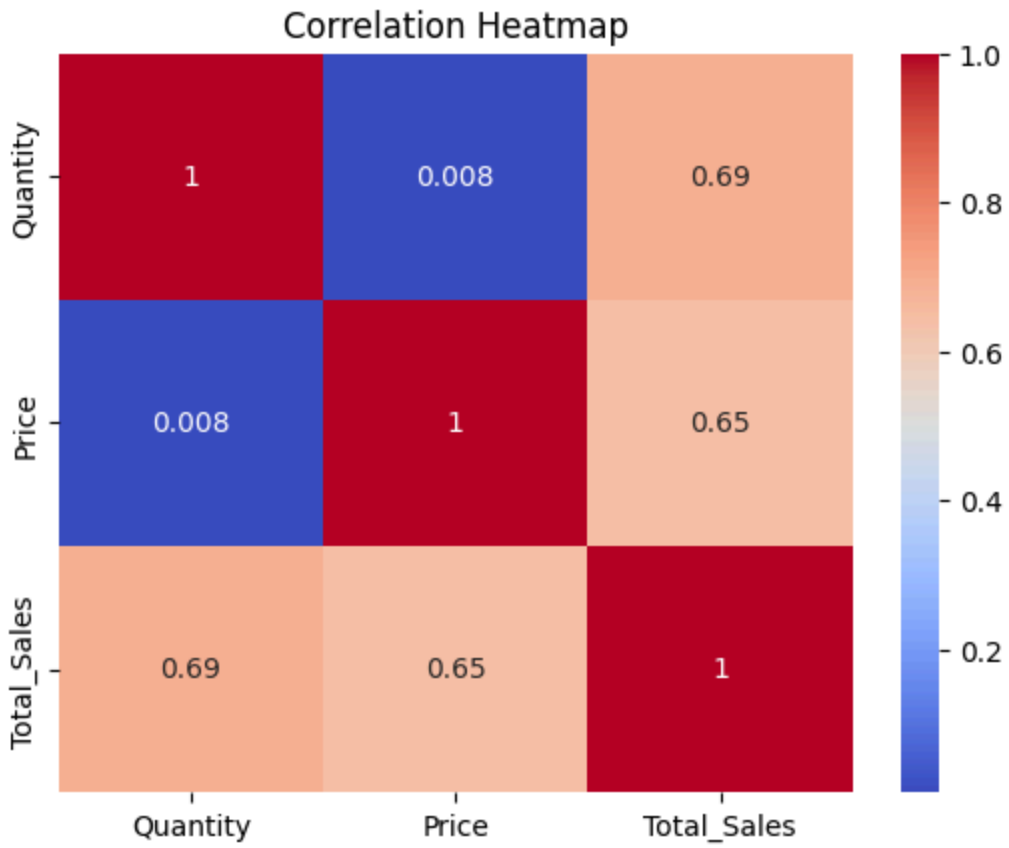
Visualizations were used to support and validate statistical findings. The following charts were generated during execution:

- Histogram of Monthly Charges

  - Helps visualize the distribution and spread of customer charges
  - Indicates skewness in customer payment behavior




● Correlation Heatmap
○ Displays relationships between tenure, monthly charges, total charges, and churn
○ Makes it easier to identify strong positive or negative correlations




These visuals complement numerical results and improve interpretability by providing intuitive graphical evidence

## Technical Details

The project applies several statistical techniques and analytical methods, explained below:

● **Descriptive Statistics**

○ Used to summarize customer tenure, monthly charges, and total charges
○ Provides an overview of central tendency and variability

● **Normality Testing**

○ Shapiro-Wilk test applied to monthly charges
○ Result indicates non-normal distribution, suggesting skewed customer spending

● **Correlation Analysis**

○ Pearson correlation used to measure linear relationships
○ Strong negative correlation observed between tenure and churn
○ Weak positive correlation observed between monthly charges and churn

● **Hypothesis Testing**

○ Independent t-test used to compare monthly charges between churned and non-churned customers
○ Independent t-test used to compare senior and non-senior customers
○ One-way ANOVA used to compare monthly charges across contract types
○ Decisions made using a significance level of 0.05

● **Confidence Interval**
○ 95% confidence interval calculated for average monthly charges
○ Provides a reliable estimate of the population mean

● **Regression Analysis**
○ Linear regression model used to predict total charges from tenure and monthly charges
○ R-squared value used to evaluate model performance

## Testing Evidence

Multiple statistical tests were conducted to validate assumptions and support conclusions:

● **Independent T-Test (Churn vs Monthly Charges)**
○ p-value < 0.05 ○ Indicates monthly charges significantly affect customer churn

● **Independent T-Test (Senior Citizen vs Non-Senior)**
○ p-value < 0.05
○ Confirms significant difference in monthly charges based on senior citizen status

● **One-Way ANOVA (Contract Type)**
○ p-value > 0.05
○ Indicates no statistically significant difference in monthly charges across contract types

● **Regression Testing**
○ R-squared value very low
○ Indicates that additional variables are required to effectively predict total charges
These tests confirm the validity of statistical conclusions and demonstrate correct application of statistical methods.

**Results Interpretation and Business Insights**

The analysis reveals several important business insights derived from statistical evidence:

● Customers with longer tenure are significantly less likely to churn
● Higher monthly charges slightly increase the probability of churn
● Senior citizens exhibit different pricing behavior, suggesting the need for targeted pricing strategies
● Contract type does not significantly impact monthly charges
● Revenue prediction requires more factors beyond tenure and monthly charges

Overall, the project demonstrates the effective use of statistical analysis to uncover actionable business insights and emphasizes the importance of data-driven decision-making in customer management and pricing strategies